

Dialect-aware chrF score^{*} with filtered n-grams

solutions 1-3 computed chrF by extracting dialect-specific and dialect-invariant parts into separate strings, then scoring each separately

concatenating non-adjacent spans creates fake n-gram boundaries which breaks the context of the original sentence

GOAL compute one single chrF score on the full, unmodified sentence while restricting which n-grams count toward it

^{*} Keep only n-grams that include at least one dialect-specific character and throw out n-grams made entirely of invariant text

1. Build a per-character dialect mask for both SK ref and SK hyp
(True = dialect-specific character, False = dialect-invariant character)

* reuse solution 2's NK-SK-hyp alignment logic

① locate candidate invariant positions from NK-SK reference

② among the candidate invariant positions keep those that also remain equal in the hypothesis
→ dialect-invariant positions were marked True

③ since we have to target the dialect-specific positions, flip every value (True $\leftrightarrow$ False)
 $\text{hyp_dialect_mask} = [\text{not } c \text{ for } c \text{ in } \text{hyp_invariant_mask}]$

2. For each n-gram order $n=1$ to 6:

- extract every character n-gram from the full ref string and full hyp string
- keep only n-grams that touches at least one dialect-specific character
- compute match count between filtered hyp n-grams and filtered ref n-grams
- compute precision and recall for each order

3. Average precision and recall across all orders

(avg P's together, avg R's together)

CHRP

CHRR

$$\text{Compute } \text{CHRF8} = \frac{(1+B^2) \text{CHRP} \cdot \text{CHRR}}{B^2 \cdot \text{CHRP} + \text{CHRR}}$$

ex1. NK ref : 나는 걸상을 뛰어넘기도 하고 책상 밑으로 기어나가기도 하면서 난로 옆까지 갔다.

SK ref : 나는 걸상을 뛰어넘기도 하고 책상 밑으로 기어나가기도 하면서 난로 옆까지 갔다.

SK hyp : 나는 걸상을 뛰어넘기도 하고 책상 밑으로 기어나가기도 하면서 난로 옆까지 갔다.

- = dialect-invariants parts
- = dialect-specific parts (Parts not dialect-invariant)

※ def evaluate_one_example (nk, sk, hyp) :

#1. create dialect-specific mask

ref_dialect_mask, hyp_dialect_mask = build_dialect_masks (nk, sk, hyp)

[False, ..., False, True, False, ..., False, True, ..., True
0, ..., 6, 7, 8, 9, ..., 15, 16, ..., 21
False, False, True, False, ..., False, True, ..., True, False, ..., False]
22, 23, 24, 25, ..., 33, 34, ..., 39, 40, 43

#2. n-grams are extracted from the full sk/hyp sentences
and only those that include at least one dialect-specific character

dialect_aware_score = filtered_chrf(
hyp, sk, hyp_dialect_mask, ref_dialect_mask,
overlap_rule = overlap_rule
)

return { "dialect_aware_chrf3_filtered" : dialect_aware_score }

#1. create dialect-specific mask

※ `def build_dialect_masks(nk, sk, hyp) → tuple[list[bool], list[bool]]:`

```
"""
1. Identify candidate invariant characters from the NK-SK pair (+ remove spacing-sensitive regions from that invariant set)
2. Retain only invariant characters that are also unchanged in the SK reference and hypothesis
3. Treat the complement as dialect-specific
"""
```

#1. locate candidate dialect-invariant positions from NK-SK reference

`initial_mask = build_initial_invariant_mask(nk, sk)`

`= [ True, ..., True, False, True, ..., True, False, ..., False,`
 0, 1...6, 7, 8, 9, ..., 15, 16, ..., 21
`True, True, False, True, ..., True, False, ..., False, True, ..., True]`
 22, 23, 24, 25, ..., 33, 34, ..., 39, 40, 43

#2. Keep only positions that also remain equal in the hypothesis

`ref_invariant_mask, hyp_invariant_mask = (
 refine_invariant_mask_with_hypothesis(sk, hyp, initial_mask)
)`

in this example, `initial_mask == ref_invariant_mask == hyp_invariant_mask`

#3. dialect masks = negation of invariant masks

`ref_dialect_mask = [ not c for c in ref_invariant_mask ]`

`hyp_dialect_mask = [ not c for c in hyp_invariant_mask ]`

`= [ False, ..., False, True, False, ..., False, True, ..., True`
 0, 1...6, 7, 8, 9, ..., 15, 16, ..., 21
`False, False, True, False, ..., False, True, ..., True, False, ..., False]`
 22, 23, 24, 25, ..., 33, 34, ..., 39, 40, 43

`return ref_dialect_mask, hyp_dialect_mask`

#2. n-grams are extracted from the full sk/hyp sentences and only those that include at least one dialect-specific character

```
※ def filtered_chrf(hyp, ref, hyp_mask, ref_mask, overlap_rule: str = "any",  
                    max_order: int = 6, beta: int = 3) → float:
```

```
    precisions, recalls = [], []
```

```
    for n in range(1, max_order + 1):
```

```
        hyp_all_ngrams = extract_char_ngrams_with_positions(hyp, n)
```

```
        ref_all_ngrams = extract_char_ngrams_with_positions(ref, n)
```

```
        hyp_filtered = filter_ngrams_by_mask(hyp_all_ngrams, hyp_mask,  
                                              overlap_rule)
```

```
        ref_filtered = filter_ngrams_by_mask(ref_all_ngrams, ref_mask,  
                                              overlap_rule)
```

```
        total_hyp = len(hyp_filtered)
```

```
        total_ref = len(ref_filtered)
```

```
        matched = clipped_match_count(hyp_filtered, ref_filtered)
```

```
        p_n = matched / total_hyp
```

```
        r_n = matched / total_ref
```

```
        precisions.append(p_n)
```

```
        recalls.append(r_n)
```

```
mean_p = sum(precisions) / len(precisions)
```

```
mean_r = sum(recalls) / len(recalls)
```

```
f_beta = ((1 + beta**2) * mean_p * mean_r  
          (beta**2 * mean_p + mean_r))
```

$$\text{CHRP} = \frac{1}{N} \sum_{n=1}^N P_n \quad - \quad \text{CHRR} = \frac{1}{N} \sum_{n=1}^N R_n$$

The general formula for the CHRF score is:

$$\text{CHRF}_\beta = (1 + \beta^2) \frac{\text{CHRP} \cdot \text{CHRR}}{\beta^2 \cdot \text{CHRP} + \text{CHRR}}$$

①

 $n = 1$ `hyp_all_ngrams = extract_char_ngrams_with_positions(hyp, 1)``ref_all_ngrams = extract_char_ngrams_with_positions(ref, 1)``hyp_filtered = filter_ngrams_by_mask(hyp_all_ngrams, hyp_mask, overlap_rule)`
`= ['어', '책', '상', '-', '밑', '여', '난', '로', '-', '옆']``ref_filtered = filter_ngrams_by_mask(ref_all_ngrams, ref_mask, overlap_rule)`
`= ['어', '책', '상', '-', '밑', '어', '난', '로', '-', '옆']``total_hyp = len(hyp_filtered) = 10``total_ref = len(ref_filtered) = 10``matched = clipped_match_count(hyp_filtered, ref_filtered) = 9` $p_1 = \text{matched} / \text{total\_hyp} = 9 / 10 = 0.9$  $r_1 = \text{matched} / \text{total\_ref} = 9 / 10 = 0.9$ `precisions = [ p_1 ]``recalls = [r_1]`

②

 $n = 2$ `hyp_all_ngrams = extract_char_ngrams_with_positions(hyp, 2)``ref_all_ngrams = extract_char_ngrams_with_positions(ref, 2)``hyp_filtered = filter_ngrams_by_mask(hyp_all_ngrams, hyp_mask, overlap_rule)`
`= [ '뛰어', '어넘', '책', '책상', '상-', '밑', '밑으', '으로', '로-',`
`'기여', '여나', '-난', '난로', '로-', '-옆', '옆까', '까지', '지-']``ref_filtered = filter_ngrams_by_mask(ref_all_ngrams, ref_mask, overlap_rule)`
`= [ '뛰어', '어넘', '책', '책상', '상-', '밑', '밑으', '으로', '로-',`
`'기어', '어나', '-난', '난로', '로-', '-옆', '옆까', '까지', '지-']``total_hyp = len(hyp_filtered) = 18``total_ref = len(ref_filtered) = 18``matched = clipped_match_count(hyp_filtered, ref_filtered) = 16` $p_1 = \text{matched} / \text{total\_hyp} = 16 / 18 = 0.88$  $r_1 = \text{matched} / \text{total\_ref} = 16 / 18 = 0.88$ `precisions = [ p_1, p_2 ]``recalls = [r_1, r_2]`

continue this process ... for $n = 3, 4, 5, 6$

```
precisions = [p_1, p_2, p_3, p_4, p_5, p_6]  
recalls = [r_1, r_2, r_3, r_4, r_5, r_6]
```

```
mean_p = sum(precisions) / len(precisions)
```

```
mean_r = sum(recalls) / len(recalls)
```

```
f_beta = ((1 + beta**2) * mean_p * mean_r)  
         (beta**2 * mean_p + mean_r)
```

```

※ def extract_char_ngrams_with_positions (text: str, n: int):
    return [(text[i:i+n], i, i+n) for i in range(len(text) - n + 1)]

```

length- n character windows of text, [start, end) offsets

Model output

나는 결상을 뛰어넘기도 하고 책상 밑으로 기어나가기도 하면서 난로 옆까지 갔다.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 23 24 25 26 27 28 39 40 41 42

① extract_char_ngrams_with_positions (hyp, 1)

for i in range(44-1+1): # i = 0 ~ 43

(text[0:1], 0, 1), (text[1:2], 1, 2) ... (text[42:43], 42, 43)

 '나' '는' '다'

... (text[43:44], 43, 44)

 '.'

② extract_char_ngrams_with_positions (hyp, 2)

for i in range(44-2+1): # i = 0 ~ 42

(text[0:2], 0, 2), (text[1:3], 1, 3) ... (text[41:43], 41, 43)

 '나는' '는 _' '갔다'

... (text[42:44], 42, 44)

 '다.'

⋮

⑥ extract_char_ngrams_with_positions (hyp, 6)

for i in range(44-6+1): # i = 0 ~ 38

(text[0:6], 0, 6), (text[1:7], 1, 7) ... (text[37:43], 37, 43)

 '나는 결상을' '는 결상을 _' '옆까지 갔다'

... (text[38:44], 38, 44)

 '까지 갔다.'

SK reference

나는 결상을 뛰어넘기도 하고 책상 밑으로 기어나가기도 하면서 난로 옆까지 갔다.

→ repeat similar process for SK reference

```
※ def filter_ngrams_by_mask (ngrams_with_positions, mask,
                                overlap_rule: str = "any") :
```

```
    kept = [ ]
```

```
    for ngram, start, end in ngrams_with_positions :
        window = mask [start : end]
        dialect_count = sum(window)
        window_len = end - start
```

```
        if overlap_rule == "any" :
            keep = dialect_count >= 1
```

```
        if keep:
            kept.append(ngram)
```

```
    return kept
```

Model output

나는 결상을 뛰어넘기도 하고 책상 밑으로 기여나가기도 하면서 난로 옆까지 갔다.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

23 24 25 26 27 28

40 41 42

① n=1

- filter_ngrams_by_mask (hyp_all_ngrams, hyp_mask, overlap_rule)

```
hyp_all_ngrams = [ (text[0:1], 0, 1), (text[1:2], 1, 2) ...
                   (text[42:43], 42, 43), (text[43:44], 43, 44) ]
```

```
hyp_mask = [ False, ..., False, True, False, ..., False, True, ..., True, False, False,
              0, 1, ..., 6, 7, 8, 9, ..., 15, 16, ..., 21, 22, 23,
              True, False, ..., False, True, ..., True, False, ..., False ]
              24, 25, ..., 33, 34, ..., 39, 40, 43
```

```
overlap_rule = "any"
```

```
kept = [ '어', '책', '상', ' ', '밑', '어', '난', '로', ' ', '옆' ]
```

- filter_ngrams_by_mask (ref_all_ngrams, ref_mask, overlap_rule)

```
kept = [ '어', '책', '상', ' ', '밑', '어', '난', '로', ' ', '옆' ]
```

2930313233343536373839

404142

kept = ['뛰어', '어넘', '_책', '책상', '상_', '_밑', '밑으', '으로', '로_', '기어', '어나',
'_난', '난로', '로_', '_옆', '옆까', '까지', '지_']

※ def clipped_match_count(hyp_ngrams, ref_ngrams) → int :

```
hyp_counter = Counter(hyp_ngrams)
ref_counter = Counter(ref_ngrams)
common = hyp_counter & ref_counter
```

```
return sum(common.values())
```

① n=1

```
matched = clipped_match_count(hyp_filtered, ref_filtered)
```

```
hyp_filtered = ['어', '책', '상', '-', '밑', '여', '난', '로', '-', '옆']
```

```
ref_filtered = ['어', '책', '상', '-', '밑', '어', '난', '로', '-', '옆']
```

```
Counter(hyp_filtered) = { '어':1, '책':1, '상':1, '-':2, '밑':1,
                          '여':1, '난':1, '로':1, '옆':1 }
```

```
Counter(ref_filtered) = { '어':2, '책':1, '상':1, '-':2, '밑':1,
                          '난':1, '로':1, '옆':1 }
```

```
common = Counter(hyp_filtered) & Counter(ref_filtered)
           ↳ takes min() per key
           = { '어':1, '책':1, '상':1, '-':2, '밑':1,
               '난':1, '로':1, '옆':1 }
```

```
sum(common.values()) = 9
```

② n=2

```
matched = clipped_match_count(hyp_filtered, ref_filtered)
```

```
hyp_filtered = ['뛰어', '어넘', '-책', '책상', '상-', '-밑', '밑으', '으로', '로-',
                 '기여', '여나', '-난', '난로', '로-', '-옆', '옆까', '까지', '지-']
```

```
ref_filtered = ['뛰어', '어넘', '-책', '책상', '상-', '-밑', '밑으', '으로', '로-',
                 '기어', '어나', '-난', '난로', '로-', '-옆', '옆까', '까지', '지-']
```

```
Counter(hyp_filtered) = { '뛰어':1, '어넘':1, '-책':1, '책상':1, '상-':1, '-밑':1,
                          '밑으':1, '으로':1, '로-':2, '기여':1, '여나':1,
                          '-난':1, '난로':1, '-옆':1, '옆까':1, '까지':1, '지-':1 }
```

```
Counter(ref_filtered) = { '뛰어':1, '어넘':1, '-책':1, '책상':1, '상-':1, '-밑':1,
                          '밑으':1, '으로':1, '로-':2, '기어':1, '어나':1,
                          '-난':1, '난로':1, '-옆':1, '옆까':1, '까지':1, '지-':1 }
```

n-gram	hyp count	ref count	min count ("clipped")
'뛰어', '어넘', '_책'	1 each	1 each	8 in total
'책상', '상_', '_밑', '밑으', '으로'			
'로_'	2	2	2
'기여' not in ref	1	0	0
'기어' not in hyp	0	1	0
'여나' not in ref	1	0	0
'어나' not in hyp	0	1	0
'_난', '난로', '_옆', '옆까', '까지', '지_'	1 each	1 each	6 in total

Sum(common.values()) = 16